CyberGuard AI

Project Survey Report

Generated: October 08, 2026

1. Project Overview

CyberGuard AI is a full-stack Security Operations Centre (SOC) platform with an AI-driven threat analysis backend and a React dashboard frontend. The project was built incrementally across multiple milestones (Steps 2–8.5.x), each adding a new detection or integration capability.

Area	Technology
Backend framework	FastAPI 0.115+ / Python 3.14 / Uvicorn
Database	PostgreSQL 16 via psycopg3 + SQLAlchemy 2.0 + Alembic
Cache / state	Redis 7
Event bus	Apache Redpanda (Kafka-compatible)
Frontend	React 18 + TypeScript, Vite 5, TanStack Query v5, Tailwind CSS
Auth	Firebase (Google OAuth / email / phone) + backend JWT
ML	scikit-learn TF-IDF + Logistic Regression (phishing classifier)
Threat intel	Google Safe Browsing API + VirusTotal + local IOC rules
Infra	Docker Compose (backend, PostgreSQL, Redis, Redpanda)

2. Repository Layout

Directory	Contents
backend/app/	FastAPI application — agents, API routes, services, ML, DB, auth
backend/tests/	~27 pytest test modules
backend/ml_models/	Trained model artefacts (phishing_model.pkl, vectorizer)
frontend/src/	React pages, components, contexts, services

docs/	Step implementation reports (STEP_*.md, GMAIL_AUTH_401_FIX_REPORT.md)
email_analysis_ml_data/	Offline ML data pipeline scripts
docker-compose.yml	Multi-service orchestration

3. Backend Modules

3.1 Agents

Agent	Status	Description
EmailAgent	Implemented	SPF/DKIM/DMARC, URL analysis, attachment scan, ML classify, brand impersonatio
LogAgent	Implemented	Windows Event Log + Linux syslog / journald collector
NetworkAgent	Stub	Placeholder — registered but returns pending
TaskDispatcher	Implemented	Routes analysis requests; falls back to PlaceholderAgent on import failure

3.2 API Routes (v1)

Route prefix	Feature
/auth	Login, Firebase exchange, JWT verify, user management
/gmail	OAuth connect/disconnect, list/get/analyze/investigate messages
/analysis	Manual email / log / network analysis submission
/detection	Rule engine, IOC matching, anomaly, scoring
/alerts	CRUD, status, statistics
/events	Ingest, reanalyze, statistics
/incidents	Create (manual + from correlation), timeline, statistics
/correlation	Cross-event correlation engine
/logs	Log source management, event ingestion, raw ingest
/reports	Summary, executive, threat-landscape, compliance
/llm	Threat analysis, IOC enrichment, MITRE mapping, remediation
/knowledge	RAG search, MITRE technique/CVE lookup
/ws	WebSocket realtime channel manager
/metrics	Prometheus-style metrics collector

/training	Feedback export, model retrain, threshold tuning

3.3 Services

Service	Description
url_analysis/service.py	URL expansion, pattern scoring, Safe Browsing, VirusTotal
email_report_builder.py	Assembles SOC-style EmailInvestigationReport from agent output
gmail/ (OAuth + parser)	Gmail OAuth2 flow, GmailParser (MIME, auth headers, URLs, attachments)
threat_intel/	IOC batch lookup, TTL-cached results
attachment_analysis/	Macro detection, archive inspection, file hash
oauth_state_store.py	Redis-backed CSRF-safe OAuth state
token_encryption.py	AES-GCM encryption for stored OAuth tokens
circuit_breaker.py	Per-service failure isolation

4. Frontend

Page	Feature
Dashboard	KPI tiles, alert feed, realtime WebSocket updates
EmailAnalysis	Manual paste analysis + Gmail OAuth integration tab
Alerts / Incidents / Events	List, filter, detail view, status update
LogAnalysis	Source management, event browser, raw ingest
NetworkAnalysis	Stub — minimal UI
Reports	Summary, executive, compliance, threat landscape
Settings	User profile, API key management
Auth pages	Login, Signup, ForgotPassword, VerifyEmail, PhoneLogin

The frontend uses a single authenticated Axios instance with a request interceptor that attaches the backend JWT from localStorage. A response interceptor clears the session on 401 only when the request carried a token (fixed 2026-09-18 to prevent race-condition session wipes during Firebase auth initialization).

5. Machine Learning Pipeline

Component	Detail
Classifier	TF-IDF vectorizer + Logistic Regression (scikit-learn)
Training data	Offline pipeline in email_analysis_ml_data/
Inference	app/ml/classifier.py — ml_predict() in EmailAgent
Continuous learning	FeedbackExporter → ModelTrainer → ThresholdTuner
Model artefacts	backend/ml_models/phishing_model.pkl + vectorizer.pkl
Ensemble	app/ml/ensemble.py — multiple model voting (if configured)

6. Test Coverage

Backend has ~27 pytest test modules. Last full run: 781 passed / 1 pre-existing Redis e2e failure / 26 skipped.

Test file	Covers
test_email_regression.py	Email agent end-to-end scenarios
test_url_analyzer.py	URL pattern + Safe Browsing + VirusTotal
test_step6_oauth_state.py	Gmail OAuth state (29 tests)
test_step8_*.py (6 files)	Correlation engine, Redis state, log ingestion, web ingest, e2e
test_step7*.py (3 files)	Event bus, Redpanda live integration
test_step5_*.py	Windows/Linux log collector
test_feedback_*.py	Continuous learning feedback loop
test_detection_*.py	Rule engine, IOC, anomaly, scoring
Frontend tests	None — no test framework configured

7. Known Issues & Incomplete Areas

Severity	Issue	Location
HIGH	serviceAccountKey.json committed to repo (Firebase credentials)	backend/
HIGH	Malformed pip artefacts: =0.10.0 and =3.4.0 directories	backend/
MEDIUM	RAG knowledge base has no vector DB dependency — may be non-functional	app/rag/
MEDIUM	LLM client requires external API key with no SDK pinned in requirements	app/llm/
MEDIUM	NetworkAgent is a stub — returns pending for all requests	app/agents/

LOW	Dead backup files: email_agent_v1_backup.py, email_agent_v2_backup.py, step7.py	app/agents/
LOW	No frontend test framework	frontend/
LOW	Live Redpanda tests require separately running instance	tests/
LOW	NetworkAnalysis and Reports pages are thin on the frontend	frontend/src/pages/

Action required: The Firebase service account key file should be removed from version control immediately and added to .gitignore. Rotate the key if the repository has ever been pushed to a remote.

8. Recent Fixes (Sept 2026)

Date	Fix	Files changed
2026-09-18	Email pipeline: dispatcher imported non-existent email_agent_v2 — dispatcher PENDING	dispatcher.py
2026-09-18	URL intelligence: url_analysis service never called in investigate endpoint	gmail.py
2026-09-18	Report builder: NOT_DETECTED used when no analysis ran (should be UNKNOWN)	email_report_builder.py
2026-09-18	Frontend: PENDING/error shown as 0-risk; confidence falsy-zero bug	EmailAnalysisPage.tsx
2026-09-18	Gmail 401: status query fired before auth initialized, wiped session	api.ts, EmailAnalysisPage.tsx

This report was auto-generated from a live codebase survey. Architecture details reflect the state of the repository as of the generation date.